

IMPLEMENTATION OF DIGITAL IMAGE FILTERS USING VERILOG HDL

submitted in partially fulfillment of the requirements for the degree of

Bachelor of Technology
in
Electronics and Communication Engineering

by

Anmol Saxena (2200970200001)
Lokesh Kumar (2200970310092)
Manish Rathore (2200970310094)

Under the Guidance of

Dr. Richa

Associate Professor

Department of Electronics and Communication Engineering

(B. Tech ECE - Accredited by NBA),

Galgotias College of Engineering and Technology, Greater Noida.

(Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)



April, 2026

DECLARATION

We hereby declare that the thesis entitled “**Implementation of Digital Image Filters using Verilog HDL**” submitted by us, for the award of the degree of *Bachelor of Technology in Electronics and Communication Engineering* to Galgotias College of Engineering and Technology, Greater Noida affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow is a record of bonafide work carried out by us under the supervision of **Dr. Richa**.

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Greater Noida

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the thesis entitled “**Implementation of Digital Image Filters using Verilog HDL**” submitted by Anmol Saxena, Lokesh Kumar, Manish Rathore Department of Electronics and Communication Engineering, Galgotias College of Engineering and Technology, Greater Noida affiliated to Dr. A.P.J. Abdul Kalam Technical University (AKTU), Lucknow, for the award of the degree of *Bachelor of Technology in Electronics and Communication Engineering*, is a record of bonafide work carried out by him under my supervision, as per AKTU code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The thesis fulfils the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Greater Noida

Date :

Signature of the Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner

External Examiner

Approved by

Head of the Department

ABSTRACT

The Implementation of Digital Image Filters using Verilog HDL is an important area in digital signal processing and hardware design. This project focuses on the design and simulation of various digital image filtering techniques using Verilog without using of FPGA hardware. The primary objective is to model and verify the functionality of image filters such as smoothing, sharpening, and edge detection filters through HDL coding and simulation tools.

In this work, the image data is processed by converting pixel values into a suitable format that can be handled within the Verilog environment. Different filtering algorithms, including mean filter and convolution-based filters, are implemented using behavioral and structural modeling techniques. The design is simulated using standard EDA tools to analyze accuracy, performance and correctness of the filter outputs. Since the project does not involve physical hardware implementation, the emphasis is placed on functional verification, timing analysis, and efficient HDL coding practices. This approach helps in understanding how image processing algorithms can be mapped into hardware description languages, which is useful for future FPGA or ASIC implementations.

The results demonstrate that Verilog HDL can effectively model digital image filters and validate their operation in a simulation environment. This project provides a strong foundation for hardware-based image processing system design and highlights the role of HDL in bridging software algorithms and hardware realization.

The system takes an input image, typically converted into grayscale pixel values, and applies different filtering operations such as reduction, noise, edge detection, and image enhancement. These operations are implemented using convolution techniques, where a kernel or mask is applied to the image pixels. Filters such as mean filter, Gaussian filter, and Sobel operator are modeled using Verilog constructs.

The design is developed using behavioral modeling and simulated using EDA tools like ModelSim or Xilinx Vivado simulator. Test benches are created to verify the correctness of the filter outputs. Since the implementation is software-based, the focus is on validating logic design, understanding data flow, and ensuring accurate results rather than hardware optimization.

ACKNOWLEDGEMENT

First of all, we would like to express our gratitude to our beloved **Chairman Shri. Suneel Galgotia and CEO Shri. Dhruv Galgotia** for providing necessary facilities to carry out and finish the project successfully. We are grateful to our **Director Dr. Brijesh Singh** for his support and encouragement.

Our special thanks goes to **Dr. Amrita Rai, Head of the Department, Electronics and Communication Engineering**, for providing an environment that encouraged us in working towards our goal and supported in completion of our project. This project gave us an opportunity to apply creative and critical thinking skills.

We would like to present our deep sense of gratitude to our project guide, **Dr. Richa Assistant Professor, Department of Electronics and Communication Engineering** who has always been a source of motivation and firm support for carrying out the project. The supervision and support that he gave truly helped in the progression of the project. His cooperation is much indeed appreciated. We are highly obliged to him for his valuable advices and moral support during research period.

Finally, our greatest and special gratitude goes to our family for their love and support.

Place: Greater Noida

Anmol Saxena

Lokesh Kumar

Date:

Manish Rathore

TABLE OF CONTENTS

Chapter No.	Content	Page No.
	Title Page	i
	Declaration	ii
	Certificate	iii
	Abstract	iv
	Acknowledgement	v
	Table of Contents	vi
	List of Figures	viii
	List of Tables	viii
1	Introduction	1
	1.1 Overview of Digital Image Processing	1
	1.2 Hardware Description Languages	2
	1.3 Image Enhancement Techniques	3
	1.3.1 Spatial Domain Enhancement	3
	1.3.2 frequency Domain Enhancement	4
	1.4 Point Operations and Filter Types	4
	1.5 Applications of Image Enhancement	6
	1.6 Project Objectives and Motivation	6
2	Literature Review	8
	2.1 Evolution of Image Processing on Hardware	8
	2.2 Software vs Hardware Implementations	8
	2.3 Review of Spatial Domain Enhancement Techniques	9
	2.4 Previous Work on Verilog-Based Image Filtering	9
	2.5 Problem Formulation and Identified Gaps	11
3	Methodology and System Design	12
	3.1 Proposed System Architecture Overview	12
	3.1.1 Top-Level Module Integration	12
	3.2 Input Data Interface and Buffering	15
	3.2.1 Software-to-Hardware Translation via Python	15
	3.2.2 Testbench Simulation Environment	15

3.3	Line Buffering Mechanism and Neighborhood Generation	17
3.3.1	Architecture of the Line Buffer	17
3.4	Image Control Logic and Data Synchronization	19
3.4.1	Buffer Tracking and State Machine Configuration	19
3.5	Mathematical Modeling of Implemented Filters	20
3.5.1	Brightness and Contrast Algorithms	21
3.5.2	Color Boost and Gamma Operations	22
3.5.3	Edge Sketch and Duotone Logic	23
4	Implementation and Experimental Setup	25
4.1	Tools Used: Xilinx Vivado and Python	25
4.1.1	Xilinx Vivado Design Suite	25
4.1.2	Python (Software Data Interface)	25
4.2	Input Image Preparation (Python RGB Extraction)	25
4.3	Verilog RTL Module Design	27
4.3.1	Top Module (imageProcessTop.v)	27
4.3.2	Control Module (imagecontrol.v)	28
4.3.3	Convolution Module (conv.v)	30
4.4	Streaming Data Processing Pipeline	31
4.5	Output Image Reconstruction	32
5	Results and Discussion	34
5.1	Simulation Environment Configuration and Setup	34
5.2	Timing and Control Signal Waveform Analysis	35
5.3	Visual Results and Filter Performance Comparison	36
5.3.1	Color Boost Implementation and Analysis	37
5.3.2	Duotone Filter Results	39
5.3.3	Edge Sketch Filter Results	40
5.3.4	Gamma Correction Results	41
5.4	Quantitative Performance Analysis	42
5.5	Advantages of the Proposed	42
6	Conclusion	43
	Future Scope	44
	Reference	47
	APPENDIX	48

LIST OF FIGURES

Figure No	Title	Page No.
Figure 1	Vivado XSim GUI showing Testbench Hierarchy and File I/O Console	34
Figure 2	Timing Waveform showing pixel_valid and pixel out synchronization	36
Figure 3	Original 512×512 Input Image (Lion)	37
Figure 4	Color Boost Filter Output Image	38
Figure 5	Duotone Filter Output Image	39
Figure 6	Edge Sketch Filter Output Image	40
Figure 7	Gamma Correction Output Image	41

LIST OF TABLE

Table No.	Title	Page No.
1.1	Comparison of CPU, GPU, and FPGA Performance	2
2.1	Summary of Literature Review Contributions	10
3.1	Top-Level Interface Signals Description	14

CHAPTER 1

Introduction

1.1 Overview of Digital Image Processing

Digital Image Processing (DIP) refers to the use of computer algorithms to perform image processing on digital images. As a subcategory or field of digital signal processing, digital image processing has many advantages over analog image processing. It allows a much wider range of algorithms to be applied to the input data and can avoid problems such as the build-up of noise and signal distortion during processing. Since images are defined over two dimensions (perhaps more), digital image processing may be modeled in the form of multidimensional systems. The generation and development of digital image processing are mainly affected by three factors: first, the development of computers; second, the development of mathematics (especially the creation and improvement of discrete mathematics theory); third, the demand for a wide range of applications in environment, agriculture, military, industry and medical science.

An image is typically defined as a two-dimensional function, $f(x,y)$, where x and y are spatial (plane) coordinates, and the amplitude of f at any pair of coordinates (x,y) is called the intensity or gray level of the image at that point. When x , y , and the amplitude values of f are all finite, discrete quantities, we call the image a digital image. The field of digital image processing refers to processing digital images by means of a digital computer. Note that a digital image is composed of a finite number of elements, each of which has a particular location and value. These elements are referred to as picture elements, image elements, pels, and pixels. Pixel is the term most widely used to denote the elements of a digital image.

Modern digital image processing involves several fundamental steps, including image acquisition, image enhancement, image restoration, color image processing, wavelets and multiresolution processing, compression, morphological processing, segmentation, representation and description, and object recognition. Among these, image enhancement is highly subjective and focuses on making an image look 'better' for a specific application or to a specific human observer. In recent years, the paradigm has shifted significantly toward hardware-accelerated image processing. While traditional software-based processing on general-purpose processors is flexible, it often fails to meet the strict latency and throughput requirements of real-time high-definition video streams. This limitation has driven the adoption of Application-Specific Integrated Circuits (ASICs) and Field Programmable Gate Arrays (FPGAs) to execute complex mathematical pixel transformations continuously and reliably.

1.2 Hardware Description Languages (HDLs) and FPGAs

Hardware Description Languages (HDLs) are specialized computer languages used to program the structure, design, and operation of electronic circuits, and most commonly, digital logic circuits. HDLs allow designers to simulate and test how digital circuits actually behave before manufacturing, which is highly critical when timing gets tricky in complex synchronous designs. The two most prominent and industry-standard HDLs are Verilog and VHDL (Very High Speed Integrated Circuits Hardware Description Language). Verilog, standardized as IEEE 1364, is syntactically similar to the C programming language, making it relatively straightforward for engineers to learn and implement. It supports design at multiple levels of abstraction, including the switch level, gate level, register-transfer level (RTL), and algorithmic level.

Field-Programmable Gate Arrays (FPGAs) are semiconductor devices that are based around a matrix of configurable logic blocks (CLBs) connected via programmable interconnects. FPGAs can be reprogrammed to desired application or functionality requirements after manufacturing. This feature distinguishes FPGAs from Application Specific Integrated Circuits (ASICs), which are custom manufactured for specific design tasks. An FPGA architecture generally consists of three major components: Configurable Logic Blocks (CLBs) which contain Look-Up Tables (LUTs) and flip-flops to implement logic functions; Programmable Interconnects which route signals between CLBs; and Input/Output (I/O) Blocks which interface with external components. Modern FPGAs, such as those provided by Xilinx (now AMD), also include dedicated hardware blocks like Digital Signal Processing (DSP) slices and Block RAM (BRAM), which are highly advantageous for image processing tasks.

Platform	Processing Paradigm	Latency	Power Consumption
CPU	Sequential / Instruction-driven	High (Non-deterministic)	Moderate to High

Platform	Processing Paradigm	Latency	Power Consumption
GPU	Massively Parallel (SIMD)	Moderate (Batch processing delays)	Very High
FPGA	Deeply Parallel / Hardware Pipelined	Ultra-Low (Deterministic)	Low

Table 1: Comparison of CPU, GPU, and FPGA Performance

Implementing image processing algorithms on FPGAs using Verilog HDL allows for the creation of deeply pipelined, highly parallel architectures. For instance, while a CPU must iterate through a 512x512 image sequentially (taking 262,144 clock cycles minimum just to fetch), an FPGA can be configured to process incoming pixel streams on every single clock edge. Tools like Xilinx Vivado provide comprehensive design environments that allow engineers to synthesize Verilog code, map it to specific hardware resources, perform rigorous timing analysis, and generate bitstreams for device configuration. This level of hardware control is what makes Verilog an essential language for high-performance visual computing.

1.3 Image Enhancement Techniques (Spatial vs. Frequency Domain)

Image enhancement techniques aim to improve the interpretability or perception of information in images for human viewers, or to provide 'better' input for other automated image processing techniques. These methods do not increase the inherent information content of the data, but they do increase the dynamic range of the chosen features so that they can be detected easily. Image enhancement techniques can be broadly divided into two major categories based on the domain in which the operations are performed: the Spatial Domain and the Frequency Domain.

1.3.1 Spatial Domain Enhancement

The term spatial domain refers to the image plane itself, and spatial domain processing methods operate directly on the pixels of the image. Spatial domain processes are denoted by the expression:

$$\mathbf{g}(\mathbf{x}, \mathbf{y}) = \mathbf{T}[\mathbf{f}(\mathbf{x}, \mathbf{y})]$$

where $f(x, y)$ is the input image,

$g(x, y)$ is the processed output image, and

T is an operator on f defined over some neighborhood of (x, y) .

The principal approach in defining a neighborhood about a point (x, y) is to use a square or rectangular subimage area centered at (x, y) . The center of the subimage is moved from pixel to pixel starting, say, at the top left corner. The operator T is applied at each location (x, y) to yield the output, g , at that location. When the neighborhood is simply a 1×1 area, the operator becomes a point processing operation.

1.3.2 Frequency Domain Enhancement

Frequency domain methods are based on modifying the Fourier transform of an image. The technique consists of computing the 2-D discrete Fourier transform (DFT) of the image, multiplying the result by a filter function, and taking the inverse transform to produce the enhanced image. Edges and sharp transitions (such as noise) in an image contribute significantly to the high-frequency content of the Fourier transform. Thus, smoothing (blurring) is achieved by attenuating a specified range of high-frequency components (Low-Pass Filtering), while sharpening is achieved by attenuating low-frequency components (High-Pass Filtering). While frequency domain techniques offer precise control over spectral components, they are mathematically intensive and require significant memory buffering, making them less ideal for low-latency, real-time hardware implementation compared to spatial domain methods.

1.4 Point Operations and Filter Types

Point operations are the simplest form of spatial domain operations. In a point operation, the output pixel's intensity is dependent entirely upon the intensity of the input pixel at the exact same spatial coordinates, irrespective of the values of neighboring pixels. They are memoryless operations. The transformation function $f(a)$ is applied to obtain the new pixel value such that $a' = f(a)$. Because point operations do not require buffering of adjacent spatial rows, they are exceptionally fast and require minimal hardware resources (like Block RAMs or FIFO buffers), making them perfectly suited for FPGA streaming architectures.

Implemented Filter Types

This project implements several advanced point operations and hardware-optimized filters utilizing Verilog HDL. Each filter relies on specific mathematical transformations of the 24-bit RGB pixel data stream:

- **Color Boost:** This filter enhances color saturation by calculating the luminance (grayscale) value of the pixel and amplifying the difference between the original color

channel and the grayscale value. The hardware implementation utilizes bit-shifting for rapid multiplication (*e.g.*, $\text{clamp}(\text{gray} + ((R - \text{gray}) \ll 1))$).

- **Duotone Filter:** This threshold-based filter reduces the image to exactly two colors (e.g., a dark tone and a light tone). The grayscale intensity of the incoming pixel is calculated and compared against a predefined threshold (DUOTONE_TH = 128). If the value exceeds the threshold, the output is assigned the light color; otherwise, it is assigned the dark color.
- **Edge Sketch Filter:** While traditional edge detection (like Sobel or Prewitt) requires a 3x3 spatial mask, a rapid 1D hardware approximation can be achieved by comparing the grayscale intensity of the current pixel with the immediately preceding pixel in the stream. If the absolute difference (diff) exceeds a defined EDGE_TH, the pixel is marked as an edge (black), otherwise it remains white.
- **Gamma Correction:** Gamma filters are non-linear operations used to encode and decode luminance to match the visual perception of the human eye. In hardware, an approximated gamma curve is implemented by adding a right-shifted fraction of the pixel value to itself, enhancing brightness dynamically in darker regions without blowing out highlights.
- **Film Fade & Cinematic Modes:** These filters apply a combination of contrast reduction and color channel shifting. For example, adding a 'Warm Shift' to the Red channel while subtracting from the Blue channel generates a vintage, cinematic appearance in real-time.

```
// FUNCTION: clamp_unsigned
// PURPOSE : Clamp 11-bit input to 8-bit unsigned range
// Prevents overflow in hardware implementation
function [7:0] clamp_unsigned;
    // ----- Inputs -----
    input [10:0] v;
    // ----- Logic -----
    begin
        if (v > 11'd255) begin
            clamp_unsigned = 8'd255; // Upper saturation
        end
        else begin
```

```

        clamp_unsigned = v[7:0]; // Pass lower 8 bits
    end
end
endfunction

```

1.5 Applications of Image Enhancement

The ability to rapidly enhance images is a cornerstone of modern digital systems. Processing these images in real-time via hardware opens up critical applications across various industries:

- **Remote Sensing and GIS Technology:** Satellite imagery and aerial photography often suffer from atmospheric scattering, sensor noise, and poor illumination. Image enhancement algorithms are strictly applied to geospatial data to clarify terrain features, monitor agricultural health, and map urban developments. Hardware-based filters allow earth observation satellites to process enormous image matrices in orbit before transmitting the compressed data back to ground stations.
- **Medical Imaging:** Modalities such as X-rays, MRIs, and CT scans require extremely high-contrast outputs to detect micro-fractures, tumors, or anomalies. Contrast stretching and edge enhancement are routinely applied to medical visuals to aid precise clinical diagnoses.
- **Industrial Machine Vision:** Automated manufacturing lines utilize high-speed cameras to inspect products for defects (e.g., PCB solder joint inspection or surface scratch detection). Real-time hardware filters are necessary to process thousands of frames per second, isolating defective products instantly without halting the production line.
- **Surveillance and Security:** CCTV cameras operating in low-light, foggy, or rainy conditions capture degraded images. Gamma correction, histogram equalization, and adaptive filtering running natively on camera FPGAs can clarify faces and license plates in real-time before the video is even recorded to the server.

1.6 Project Objectives and Motivation

The core motivation behind this project is to address the computational bottlenecks associated with software-based image processing by shifting the processing burden to dedicated digital hardware. Traditional software executes instructions sequentially, leading to high latency and frame-rate drops when applying complex filters to high-resolution images. By utilizing the Verilog Hardware Description Language on the Xilinx Vivado platform, this project aims to create a deeply pipelined architecture that achieves true parallel processing.

The specific objectives of this project are as follows:

1. To design and implement a streaming pixel architecture capable of handling 512x512 RGB image data using AXI-style valid/ready handshake protocols.
2. To develop a modular hardware framework consisting of an Image Control Unit, Line Buffers, and an Arithmetic Convolution Module.
3. To successfully program multiple spatial domain enhancement techniques, including Color Boost, Duotone, Edge Sketch, and Gamma Correction, strictly at the Register-Transfer Level (RTL).
4. To simulate the hardware logic using a testbench environment, validating the synchronization of pixel streams, clock cycles, and output accuracy.
5. To verify the visual output by converting the original image to RGB text data via Python, passing it through the simulated Verilog pipeline, and reconstructing the output text data back into an enhanced image.

By successfully achieving these objectives, the project demonstrates the high efficiency, scalability, and deterministic latency of FPGA-based image processing systems, paving the way for implementations in embedded vision systems and real-time high-definition video processing.

CHAPTER 2

Literature Review

2.1 Evolution of Image Processing on Hardware

The domain of digital image processing has undergone a significant transformation since its inception, heavily influenced by the exponential growth of computational hardware. Early implementations of image processing algorithms relied almost exclusively on general-purpose computers and sequential microprocessors. As detailed by Russ in the *"Image Processing Handbook"* [1], the fundamental principles of image processing were initially constrained by the memory and speed limitations of early computational architectures. Processing even moderately sized images required substantial time, rendering real-time applications virtually impossible. The subsequent introduction of Digital Signal Processors (DSPs) offered an intermediate solution, providing specialized instruction sets for multiply-accumulate (MAC) operations that are ubiquitous in convolution and filtering tasks. However, as sensor resolutions increased from standard definition to high definition and beyond, even the most advanced DSPs faced processing bottlenecks. Burger and Burge, in *"Principles of Digital Image Processing - Fundamental Techniques"* [3], highlighted that the transition toward hardware-centric processing became an absolute necessity for modern visual computing, setting the stage for the widespread adoption of Application-Specific Integrated Circuits (ASICs) and Field Programmable Gate Arrays (FPGAs).

2.2 Software vs. Hardware Implementations (Why Verilog?)

The dichotomy between software-based and hardware-based image processing is fundamentally rooted in execution architecture. Software implementations, typically developed in high-level languages like C++ or Python (often utilizing libraries such as OpenCV) and executed on CPUs or GPUs, operate on a sequential instruction execution model. While GPUs offer massive parallelism through Single Instruction, Multiple Data (SIMD) architectures, they still suffer from inherent latencies associated with memory access, bus transfers, and operating system overhead. In stark contrast, FPGA implementations utilizing Hardware Description Languages like Verilog construct dedicated physical data paths for specific algorithms.

This hardware approach allows for deeply pipelined architectures where multiple stages of an algorithm execute concurrently on different pixels at every clock cycle. Zuloaga et al. [4] demonstrated the critical importance of robust hardware verification environments by proposing specialized test benches for digital image processing systems, emphasizing that the reliability of hardware processing vastly outperforms software equivalents in deterministic,

mission-critical scenarios. Verilog, specifically, has emerged as the premier HDL for such applications due to its syntactic accessibility, excellent synthesis translation, and dominant presence in modern VLSI design workflows. By defining circuits at the Register-Transfer Level (RTL), engineers can precisely manage clock synchronization, memory buffers, and logical operations, achieving real-time throughput that software paradigms inherently cannot match.

2.3 Review of Spatial Domain Enhancement Techniques

A critical component of this project's foundation lies in understanding the mathematical models governing image enhancement. Maini and Aggarwal [2], in their comprehensive review of image enhancement techniques, meticulously categorized these methods into spatial and frequency domains. They concluded that spatial domain operations—where transformations are applied directly to the image matrix—are profoundly more suitable for real-time hardware implementations due to their avoidance of computationally intensive global transforms (such as the Fast Fourier Transform).

Spatial domain methods encompass point operations (like contrast stretching and gamma correction) and neighborhood operations (like convolution for edge detection). The literature overwhelmingly supports point operations for streaming architectures because they do not strictly require massive frame buffers; the output pixel is a direct mathematical function of the corresponding input pixel. This drastically reduces the reliance on external memory architectures like DDR SDRAM, confining memory usage strictly to internal block RAMs or FIFO registers, thereby maintaining ultra-low latency.

2.4 Previous Work on Verilog-Based Image Filtering

Considerable research has been directed toward porting standard image processing algorithms onto FPGA platforms utilizing Verilog HDL. The following studies form the core foundational framework for the proposed architecture:

- **Core FPGA Implementation:** Patel et al. [5] detailed the successful implementation of fundamental image enhancement algorithms on an FPGA using Verilog, demonstrating significant processing speedups over software MATLAB models. Similarly, Kumar et al. [6] provided an extensive analysis of FPGA implementations of enhancement techniques, proving that FPGAs offer the optimal balance of power consumption, cost, and performance for spatial filters.
- **Methodological Approaches:** Chiuchisan and Cerlinca [7] proposed specific methodological approaches using Verilog HDL for image enhancement, emphasizing

the necessity of modular design. They highlighted that isolating the control logic from the datapath arithmetic drastically simplifies the debugging process and allows for highly scalable designs where new filters can be added without overhauling the top-level architecture.

- **High-Level Synthesis vs. RTL:** Yahia [8] explored high-level design methodologies for image processing using Xilinx Accel DSP. While high-level synthesis (HLS) accelerates the design time by compiling C-code to HDL, the research indicated that pure RTL designs written natively in Verilog often yield superior resource utilization and tighter timing closures, justifying the manual Verilog coding approach adopted in this project.
- **Application-Specific Hardware Architectures:** The viability of these systems in critical domains was established by Dakre [9], who reviewed image enhancement hardware co-simulation explicitly for biomedical applications. Dakre concluded that hardware acceleration is vital for modalities like MRI and CT scans, where real-time contrast adjustments can significantly aid clinical diagnosis. Furthermore, Gawhane et al. [10] presented the design of a complete digital image processor using Verilog, validating the concept of integrating multiple image enhancement filters into a single, cohesive System-on-Chip (SoC) architecture.

Author(s) [Ref]	Focus Area	Key Findings / Contributions
Maini & Aggarwal [2]	Spatial vs Frequency Domains	Spatial domain operations are fundamentally better suited for fast, low-latency applications compared to complex frequency transforms.
Zuloaga et al. [4]	Hardware Verification	Proposed robust VHDL/HDL testbench setups necessary for verifying digital image pipelines prior to silicon deployment.
Patel et al. [5]	FPGA Implementations	Demonstrated raw acceleration of standard enhancement filters on FPGA hardware utilizing Verilog.

Author(s) [Ref]	Focus Area	Key Findings / Contributions
Chiuchisan et al. [7]	Modular Verilog Design	Emphasized scalable architectures in Verilog for applying sequential image filters efficiently.
Gawhane et al. [10]	Digital Image Processors	Successfully integrated multiple enhancement modules into a unified Verilog processor architecture.

Table 2 : Summary of Literature Review Contributions

2.5 Problem Formulation and Identified Gaps

Despite the extensive research presented in the literature regarding FPGA-based image processing, several operational gaps remain. Most existing implementations focus extensively on isolated, theoretical filter algorithms (e.g., standard mean or median filters) that rely heavily on off-chip SRAM or SDRAM to buffer entire image frames before processing can commence. This frame-buffer approach introduces significant latency and memory overhead, violating the strict real-time constraints required by modern continuous streaming applications.

The fundamental problem formulated for this project is the design of a highly optimized, purely streaming pixel architecture that eliminates the need for full-frame memory buffering. By implementing AXI-style valid/ready handshake protocols and isolated internal line buffers, the system must process sequential RGB pixel data continuously as it is received. Furthermore, there is a distinct lack of hardware implementations focusing on aesthetic and cinematographic enhancement filters (such as Color Boost, Duotone, and Film Fade) which require complex, non-linear cross-channel mathematical calculations (e.g., extracting grayscale luminance from RGB vectors concurrently to manipulate saturation). This project aims to bridge these gaps by developing a completely modular Verilog datapath capable of applying multiple, selectable cinematic and spatial enhancement filters in real-time on a continuous 512x512 RGB pixel stream.

CHAPTER 3

Methodology and System Design

3.1 Proposed System Architecture Overview

The primary objective of the proposed system architecture is to establish a highly efficient, real-time image processing pipeline capable of executing spatial domain enhancement algorithms directly on digital hardware. To achieve this, the architecture leverages a Field Programmable Gate Array (FPGA) platform, utilizing Verilog Hardware Description Language (HDL) to synthesize the logical data paths. Unlike traditional software-based paradigms that process image data by storing entire frames in memory (frame-buffering) and iterating through them sequentially, the proposed methodology adopts a continuous pixel-streaming architecture. This approach processes data 'on-the-fly' as it is received from the sensor or input buffer, drastically reducing memory overhead and minimizing latency to a few clock cycles.

The top-level architecture is distinctly modular, separating the control logic, memory buffering, and arithmetic operations into isolated Verilog modules. This modularity not only simplifies debugging and timing closure within the Xilinx Vivado environment but also allows for the seamless integration of additional enhancement filters without altering the fundamental data flow. The complete pipeline is encapsulated within the `imageProcessTop` module, which coordinates the interaction between the slave input interface, the internal processing engines, and the master output interface.

3.1.1 Top-Level Module Integration

The `imageProcessTop` module acts as the central wrapper for the entire architecture. It accepts a standard 8-bit streaming pixel input (`i_data`) synchronized with a valid signal (`i_data_valid`), conforming to AXI4-Stream-like handshake protocols. The module instantiates three primary sub-components: the Image Control unit (`imageControl`), the Convolution/Filter unit (`conv`), and an Output Buffer (`outputBuffer`).

```
// Top-Level Module: imageProcessTop.v
```

```
module imageProcessTop(
```

```
    input  axi_clk,
```

```
    input  axi_reset_n,
```

```
    // Slave interface (Input Stream)
```

```
    input  i_data_valid,
```

```
    input [7:0] i_data,
```

```

    output o_data_ready,
    // Master interface (Output Stream)
    output o_data_valid,
    output [7:0] o_data,
    input i_data_ready,
    // Interrupt
    output o_intr
);

wire [71:0] pixel_data;
wire pixel_data_valid;
wire axis_prog_full;
wire [7:0] convolved_data;
wire convolved_data_valid;

assign o_data_ready = !axis_prog_full;

// Instantiation of the Control Module
imageControl IC(
    .i_clk(axi_clk),
    .i_rst(!axi_reset_n),
    .i_pixel_data(i_data),
    .i_pixel_data_valid(i_data_valid),
    .o_pixel_data(pixel_data),
    .o_pixel_data_valid(pixel_data_valid),
    .o_intr(o_intr)
);

// Instantiation of the Convolution Module
conv conv(
    .i_clk(axi_clk),
    .i_pixel_data(pixel_data),
    .i_pixel_data_valid(pixel_data_valid),
    .o_convolved_data(convolved_data),

```

```

.o_convolved_data_valid(convolved_data_valid)
);

// Instantiation of the Output Buffer
outputBuffer OB (
.s_aclk(axi_clk),
.s_aresetn(axi_reset_n),
.s_axis_tvalid(convolved_data_valid),
.s_axis_tready(),
.s_axis_tdata(convolved_data),
.m_axis_tvalid(o_data_valid),
.m_axis_tready(i_data_ready),
.m_axis_tdata(o_data),
.axis_prog_full(axis_prog_full)
);
endmodule

```

Table 3.1: Top-Level Interface Signals

Signal Name	Direction	Width	Description
axi_clk	Input	1-bit	Global system clock driving all synchronous elements.
axi_reset_n	Input	1-bit	Active-low asynchronous reset signal.
i_data_valid	Input	1-bit	Indicates the presence of valid pixel data on the input bus.
i_data	Input	8-bit	Sequential 8-bit pixel data channel (R, G, or B).

Signal Name	Direction	Width	Description
o_data_valid	Output	1-bit	Indicates that the processed data is ready for the master.
o_data	Output	8-bit	Filtered output pixel data stream.

Table 3 : Top-Level Interface Signals Description

3.2 Input Data Interface and Buffering

Digital images in their raw form are inherently two-dimensional arrays of structural color data. However, hardware communication protocols—specifically those targeting continuous streams—operate sequentially in a one-dimensional flow. Therefore, a robust data interface mechanism is required to translate standard image formats (such as JPEG or PNG) into a compatible stream of RGB bytes.

3.2.1 Software-to-Hardware Translation via Python

To bridge the gap between software images and Verilog hardware simulation, a pre-processing script utilizing Python is employed. This script extracts the specific spatial dimensions of the input image (standardized to 512x512 pixels for this architecture). It systematically isolates the Red, Green, and Blue (RGB) intensity values for each spatial coordinate (x,y) and transcribes these 8-bit values sequentially into a structured text file (image_rgb.txt). Each line in this text file contains the decimal or hexadecimal representation of the RGB vector, effectively flattening the 2D image array into a 1D stream suitable for FPGA ingestion.

3.2.2 Testbench Simulation Environment

Within the Xilinx Vivado simulation environment, a comprehensive Verilog Testbench (tb.v) acts as the surrogate hardware sensor. It employs system tasks such as \$fopen to access the Python-generated image_rgb.txt file and reads the RGB values sequentially using \$fscanf on every positive clock edge. The testbench handles the assertion of the pixel_valid signal, mimicking the behavior of a real-time camera interface.

// Testbench File I/O and Streaming Simulation (tb.v)

initial begin

clk = 0;

```

rst = 1;
pixel_valid = 0;
pixel_in = 24'd0;
pixel_count = 0;
#50;
rst = 0;

// Open file generated by Python script
infile = $fopen("image_rgb.txt", "r");
outfile = $fopen("image_rgb_out.txt", "w");

// Feed pixels to the DUT (Device Under Test)
while (!$feof(infile) && pixel_count < TOTAL_PIXELS) begin
    @(posedge clk);
    if ($fscanf(infile, "%d %d %d\n", r, g, b) == 3) begin
        pixel_in  <= {r[7:0], g[7:0], b[7:0]};
        pixel_valid <= 1'b1;
        pixel_count <= pixel_count + 1;
    end else begin
        pixel_valid <= 1'b0;
    end
end

// Allow pipeline to flush residual data
@(posedge clk);
pixel_valid <= 0;
repeat (20) @(posedge clk);
$fclose(infile);
$fclose(outfile);
$finish;
end

// Capture output pixels back to text for Python reconstruction
always @(posedge clk) begin

```



```

    if (pixel_out_valid) begin
        $fwrite(outfile, "%0d %0d %0d\n", pixel_out[23:16], pixel_out[15:8],
pixel_out[7:0]);
    end
end

```

The processed output is similarly captured by the testbench and written to an output file (image_rgb_out.txt). A reverse Python script is then utilized to reconstruct these enhanced sequential pixels back into a standard two-dimensional image format for visual evaluation.

3.3 Line Buffering Mechanism and Neighborhood Generation

Spatial domain filtering—particularly operations like edge detection and localized blurring—requires simultaneous access to a defined neighborhood of pixels surrounding a central point (e.g., a 3x3 matrix). Because the incoming data stream presents pixels strictly one by one, row by row, it is impossible to evaluate a 3x3 neighborhood utilizing only instantaneous data. Storing the entire image in external memory is inefficient. Thus, internal **Line Buffers** are employed to temporarily hold consecutive rows of the image directly within the FPGA's localized Block RAM (BRAM) or distributed logic.

3.3.1 Architecture of the Line Buffer

The system implements dedicated line buffer modules (lineBuffer.v), each configured to hold exactly one complete row of the image. For a 512x512 image, the buffer depth is set to 512. The buffer acts as an intelligent First-In-First-Out (FIFO) queue utilizing distinct write (wrPntr) and read (rdPntr) pointers.

```
// Line Buffer Module: linebuffer.v
```

```

module lineBuffer(
    input  i_clk,
    input  i_rst,
    input [7:0] i_data,
    input  i_data_valid,
    output [23:0] o_data,
    input i_rd_data
);

```

```

reg [7:0] line [511:0]; // 512-byte deep line buffer array
reg [8:0] wrPntr;
reg [8:0] rdPntr;

// Write Operation
always @(posedge i_clk) begin
    if(i_data_valid)
        line[wrPntr] <= i_data;
end

always @(posedge i_clk) begin
    if(i_rst)
        wrPntr <= 'd0;
    else if(i_data_valid)
        wrPntr <= wrPntr + 'd1;
end

// Concurrent Read Operation yielding a 3-pixel neighborhood window
assign o_data = {line[rdPntr], line[rdPntr+1], line[rdPntr+2]};

always @(posedge i_clk) begin
    if(i_rst)
        rdPntr <= 'd0;
    else if(i_rd_data)
        rdPntr <= rdPntr + 'd1;
end
endmodule

```

The critical feature of this line buffer design is its continuous assignment output:

$$\text{assign } o_data = \{line[rdPntr], line[rdPntr + 1], line[rdPntr + 2]\}$$

This logic fetches three contiguous pixels horizontally in a single clock cycle, forming a 24-bit output vector. By instantiating multiple parallel line buffers, the control logic can align the outputs vertically, successfully extracting an entire spatial matrix necessary for advanced convolution algorithms without stalling the continuous data stream.

3.4 Image Control Logic and Data Synchronization

Routing sequential data into parallel line buffers and determining precisely when to trigger the convolution calculations requires an intricate state machine. The `imageControl.v` module acts as the orchestrator of the entire datapath. It manages multiple counters to track exactly which spatial coordinate (x,y) is currently entering the pipeline, which line buffer should be written to, and when a sufficient number of rows have been buffered to begin issuing valid read requests.

3.4.1 Buffer Tracking and State Machine Configuration

The control module utilizes a 2-bit counter (`currentWrLineBuffer`) to cycle through four parallel line buffers. As the 9-bit `pixelCounter` reaches the end of a 512-pixel row, the write pointer increments, routing the subsequent row of the image into the next available line buffer. Simultaneously, a `totalPixelCounter` tracks the absolute volume of data ingested.

The read mechanism is governed by a Finite State Machine (FSM) comprising two primary states: `IDLE` and `RD_BUFFER`. The system remains in `IDLE` until exactly three rows of the image (1536 pixels) have been loaded. At this threshold, the FSM transitions to `RD_BUFFER`, asserting the `rd_line_buffer` flag, thereby signaling the downstream convolution units that valid spatial neighborhoods are now available for processing.

```
// FSM inside imagecontrol.v
localparam IDLE = 1'b0, RD_BUFFER = 1'b1;
always @(posedge i_clk) begin
    if(i_rst) begin
        rdState <= IDLE;
        rd_line_buffer <= 1'b0;
        o_intr <= 1'b0;
    end else begin
        case(rdState)
            IDLE: begin
                o_intr <= 1'b0;
                // Wait for 3 complete rows (3 * 512 = 1536)
                if(totalPixelCounter >= 1536) begin
                    rd_line_buffer <= 1'b1;
                    rdState <= RD_BUFFER;
                end
            end
        endcase
    end
end
```

```

        end
    end
    RD_BUFFER: begin
        // Read until row is complete
        if(rdCounter == 511) begin
            rdState <= IDLE;
            rd_line_buffer <= 1'b0;
            o_intr <= 1'b1; // Trigger interrupt indicating row completion
        end
    end
endcase
end
end

```

Furthermore, the controller implements multiplexer logic to dynamically rotate which line buffers represent the "top", "middle", and "bottom" rows of the current 3x3 sliding window, concatenating them into a massive 72-bit output bus

$$(o_pixel_data = \{lb2data, lb1data, lb0data\})$$

that feeds directly into the arithmetic convolution module.

3.5 Mathematical Modeling of Implemented Filters

The core computational engine of the architecture resides within the Convolution and Filter Processing module (conv.v). This parameterizable block takes the synchronized pixel data and applies continuous mathematical transformations based on a selected FILTER_MODE. The operations execute concurrently on the separate Red, Green, and Blue 8-bit channels. Since an 8-bit color channel can only represent values from 0 to 255, arithmetic overflow and underflow are critical hardware concerns. The module implements strict combinatorial clamping functions to ensure data integrity.

```

// Hardware Clamping Functions to prevent arithmetic wrapping
function [7:0] clamp_signed;
    input signed [10:0] v;
    begin
        if (v < 0) clamp_signed = 8'd0;
    end
endfunction

```

```

        else if (v > 255) clamp_signed = 8'd255;
        else clamp_signed = v[7:0];
    end
endfunction
function [7:0] clamp_unsigned;
    input [10:0] v;
    begin
        if (v > 255) clamp_unsigned = 8'd255;
        else clamp_unsigned = v[7:0];
    end
endfunction

```

3.5.1 Brightness and Contrast Algorithms

Brightness Alteration (MODE_BRIGHTNESS): Brightness adjustment is a fundamental point operation. It shifts the entire histogram of the image by adding or subtracting a constant scalar value C from each individual color channel. Mathematically, for an input pixel $P_{in}(x,y)$, the output $P_{out}(x,y)$ is calculated as:

$$P_{out}(x,y) = P_{in}(x,y) + C$$

In hardware, C is defined as a signed 9-bit parameter (BRIGHT_VAL = -9'sd80). Addition is executed in a single clock cycle, followed immediately by the clamp_signed function to prevent intense white pixels from overflowing back to absolute black (0).

High Contrast (MODE_HIGH_CONTRAST): Contrast stretching attempts to expand the dynamic range of the mid-tones while crushing extreme lows and highs. The algorithm evaluates the pixel against a median threshold (128). If the pixel is below 128, its intensity is halved (darkening the darks). If it is above 128, the difference is doubled and added back to the median (brightening the lights). In Verilog, multiplication and division are optimized by replacing them with logical bit-shifts ($>> 1$ for division by 2, and $<< 1$ for multiplication by 2).

// High Contrast Logic implementation using bit-shifting

```

function [7:0] contrast_fn;
    input [7:0] x;
    reg [8:0] temp;
    begin

```

```

    if (x < 8'd128) begin
        contrast_fn = x >> 1; // Halve dark values
    end else begin
        temp = 9'd128 + ((x - 8'd128) << 1); // Double the difference for bright values
        if (temp > 9'd255) contrast_fn = 8'd255;
        else contrast_fn = temp[7:0];
    end
end
endfunction

```

3.5.2 Color Boost and Gamma Operations

Color Boost (MODE_COLOR_BOOST): To artificially enhance color saturation, the filter must determine the 'distance' of a specific color channel from pure grayscale. The system first derives the perceived luminance (grayscale) of the RGB vector using the industry-standard NTSC weighted formula:

$$Y = 0.30R + 0.59G + 0.11B$$

Because FPGAs do not natively support fast floating-point arithmetic without massive logic utilization, the system implements this mathematically as an integer fraction. The values are multiplied by whole constants and then divided by 100:

$$\begin{aligned} \text{wire [15:0]} \text{ gray_cb_full} &= (R * 8'd30) + (G * 8'd59) + (B * 8'd11) \\ \text{wire [7:0]} \text{ gray_cb} &= \text{gray_cb_full} / 8'd100 \end{aligned}$$

Once the baseline luminance (gray_cb) is calculated, the deviation of each original color channel from this baseline is doubled (using an arithmetic left shift <<< 1) and added back to the grayscale value. This pushes saturated colors further out while leaving neutral gray pixels completely unaffected.

Gamma Filter (MODE_GAMMA): True gamma correction requires calculating exponents

$$(\$V_{\{out\}} = V_{\{in\}}^{\gamma})$$

which is highly expensive in hardware. The proposed architecture implements a highly optimized linear approximation of gamma correction by scaling the input channel based on a parameterization level (GAMMA_LEVEL) and summing it with the original value:

$$\text{wire [8:0]} Rg = \{1'b0, R\} + (\{1'b0, R\} >> \text{GAMMA_LEVEL})$$

This shifts the intensity curve upward, specifically pulling details out of deep shadows without requiring multiplier instantiation.

3.5.3 Edge Sketch and Duotone Logic

Duotone Mapping (MODE_DUOTONE): This segmentation filter transforms an image into a two-color matrix based strictly on luminance thresholding. The system calculates a simplified, unweighted grayscale average ($gray = (R + G + B)/3$). The combinational logic simply evaluates ($gray > DUOTONE_TH$). If the condition evaluates to true, the output is clamped to a predefined 24-bit COLOR_LIGHT hexadecimal value; if false, it clamps to COLOR_DARK.

Edge Sketch Filter (MODE_EDGE_SKETCH): While standard architectures employ a full 3x3 Sobel matrix for edge detection, the proposed streaming architecture implements an ultra-low-latency 1-Dimensional edge detector to create a sketched effect. As data flows through the pipeline, a register (prev_pixel) continually stores the exact value of the pixel from the previous clock cycle (\$x-1\$).

```
// 1D Edge Sketch continuous assignment logic
wire [9:0] gray_duo = R + G + B;
wire [7:0] gray = gray_duo / 3;

wire [9:0] gray_prev_sum = Rp + Gp + Bp;
wire [7:0] gray_prev = gray_prev_sum / 3;
// Calculate absolute difference between adjacent pixels
wire [7:0] diff = (gray > gray_prev) ? (gray - gray_prev) : (gray_prev - gray);

// Edge Evaluation (in the always block)
MODE_EDGE_SKETCH: begin
    pixel_out <= (diff > EDGE_TH) ? 24'h000000 : 24'hFFFFFF;
end
```

The system computes the absolute difference in luminance between the current pixel and its immediate horizontal predecessor. If this discrete derivative (diff) exceeds an edge tolerance threshold ($EDGE_TH = 4$), a hard structural boundary is detected, and the pixel is driven to absolute black (24'h000000). Otherwise, the background is driven to pure white (24'hFFFFFF). This creates a high-contrast pencil sketch effect using only

comparators and subtraction logic, completely eliminating the need for complex matrix multiplication and reducing the critical path delay of the design.

CHAPTER 4

Implementation and Experimental Setup

4.1 Tools Used: Xilinx Vivado and Python

The physical implementation and validation of the proposed hardware architecture require a hybrid software-hardware co-simulation environment. This environment bridges the gap between high-level image data formats and low-level digital logic execution. The primary tools selected for this project are Xilinx Vivado Design Suite for hardware synthesis and simulation, and Python for data pre-processing and post-processing.

4.1.1 Xilinx Vivado Design Suite: Vivado is an integrated design environment (IDE) created by Xilinx (now AMD) for the synthesis and analysis of HDL designs. It replaces the older Xilinx ISE. Vivado was selected for this project due to its highly advanced logic synthesis engine, its robust integration with AXI4-Stream protocols, and its native simulation capabilities (XSim). Within Vivado, the entire datapath—from the input buffer to the convolution core and output buffer—was coded at the Register-Transfer Level (RTL) using Verilog. Vivado's elaboration tools allow for the visualization of the inferred RTL schematics, ensuring that the written Verilog correctly translates into the intended multiplexers, adders, and line buffers rather than inefficient sequential logic. Furthermore, Vivado's simulation environment is critical for verifying the cycle-accurate behavior of the `'pixel_valid'` and `'pixel_out_valid'` synchronization signals across the pipeline.

4.2.2 Python (Software Data Interface): While Verilog is exceptional at manipulating bitstreams, it cannot natively read compressed image formats such as JPEG or PNG. Python, equipped with the Pillow (PIL) and NumPy libraries, serves as the critical data interface. Python is used to deconstruct the visual image into an uncompressed, one-dimensional stream of raw RGB bytes formatted as a text file. This text file acts as the stimulus for the Vivado testbench. Once Vivado completes the simulation and outputs the processed bitstream to a new text file, Python is utilized again to reconstruct the text data back into a viewable graphical image format. This hybrid approach ensures that the Verilog modules are tested with real-world spatial data rather than randomized test vectors.

4.2 Input Image Preparation (Python RGB Extraction)

To feed the hardware pipeline, the original input image must be converted into a sequential, line-by-line data stream that mimics the behavior of a digital image sensor. The standard test image utilized in this project is a 512x512 pixel color image. Since each pixel contains Red, Green, and Blue channels, and each channel is 8 bits (1 byte), a single pixel constitutes 24 bits of data.

The Python extraction script performs the following sequential operations:

1. **Image Ingestion:** The ``Image.open()`` function from the PIL library reads the source image file.
2. **Resizing and Normalization:** If the image is not exactly 512x512, it is resized to match the hardware line buffer depth. It is also ensured that the image is in the 'RGB' color space, stripping any Alpha (transparency) channels.
3. **Matrix Flattening:** The script iterates through the image array row by row (from $y=0$ to 511, and $x=0$ to 511). At each coordinate, it extracts the integer values for R, G, and B (each ranging from 0 to 255).
4. **Text Generation:** The extracted integer values are written to a text file named `image_rgb.txt`. Each line of the text file contains exactly one pixel's data, formatted as three space-separated integers (e.g., 255 204 106). This results in a text file with exactly 262,144 lines ($512 * 512$).

Python Pseudo-code for Input Image Preparation

```
from PIL import Image
```

```
def image_to_text(image_path, output_path):
```

```
    img = Image.open(image_path).convert('RGB')
```

```
    img = img.resize((512, 512))
```

```
    pixels = img.load()
```

```
    with open(output_path, 'w') as f:
```

```
        for y in range(img.height):
```

```
            for x in range(img.width):
```

```
                r, g, b = pixels[x, y]
```

```
                # Write R, G, B values separated by space
```

```
                f.write(f"{r} {g} {b}\n")
```

```
image_to_text("input_lion.jpg", "image_rgb.txt")
```

This purely sequential file perfectly emulates the streaming nature of hardware interfaces like AXI4-Stream, allowing the Vivado testbench to read one line per clock cycle using the \$fscanf Verilog system task.

4.3 Verilog RTL Module Design

The hardware design is implemented using a strictly modular Register-Transfer Level (RTL) approach. The system is divided into three primary hierarchical layers: the Top Module (integration), the Control Module (memory and synchronization), and the Convolution Module (arithmetic logic).

4.3.1 Top Module (imageProcessTop.v)

The imageProcessTop module serves as the primary wrapper for the system. It defines the external interfaces and instantiates the internal sub-modules, wiring them together. The interface is designed to be compatible with AXI-Stream handshaking, utilizing valid and ready signals alongside the data bus.

```
module imageProcessTop(  
    input  axi_clk,  
    input  axi_reset_n,  
    // Slave interface (Input from Sensor/Testbench)  
    input  i_data_valid,  
    input  [7:0] i_data,  
    output o_data_ready,  
    // Master interface (Output to Display/Testbench)  
    output o_data_valid,  
    output [7:0] o_data,  
    input  i_data_ready,  
    // Interrupt  
    output o_intr  
);  
wire [71:0] pixel_data;  
wire pixel_data_valid;  
wire axis_prog_full;  
wire [7:0] convolved_data;
```

```

wire convolved_data_valid;
assign o_data_ready = !axis_prog_full;
// Control Module Instantiation
imageControl IC(
    .i_clk(axi_clk),
    .i_rst(!axi_reset_n),
    .i_pixel_data(i_data),
    .i_pixel_data_valid(i_data_valid),
    .o_pixel_data(pixel_data),
    .o_pixel_data_valid(pixel_data_valid),
    .o_intr(o_intr)
);
// Convolution Module Instantiation
conv conv(
    .i_clk(axi_clk),
    .i_pixel_data(pixel_data),
    .i_pixel_data_valid(pixel_data_valid),
    .o_convolved_data(convolved_data),
    .o_convolved_data_valid(convolved_data_valid)
);
// Output buffer logic omitted for brevity...
endmodule

```

The critical pathway here is the 72-bit pixel_data bus connecting the imageControl module to the conv module. Because spatial filtering requires a 3x3 pixel neighborhood, the control module must output 9 pixels simultaneously (9 pixels * 8 bits = 72 bits). This wide data bus enables the convolution unit to perform spatial calculations in a single clock cycle without waiting for sequential pixel fetches.

4.3.2 Control Module (imagecontrol.v)

The Control Module is the most complex sequential logic block in the design. Its primary responsibility is to manage the flow of the 1D input stream into multiple parallel 1D line buffers, thereby constructing the 2D spatial neighborhood required for filtering.

The module instantiates four distinct lineBuffer units. As pixel data arrives (indicated by i_pixel_data_valid), a 9-bit pixelCounter increments from 0 to 511. Once a full row of 512 pixels is written, a 2-bit counter (currentWrLineBuffer) increments, redirecting the subsequent row of incoming pixels to the next available line buffer. This acts as a circular buffer array.

```
// FSM for Read Control inside imagecontrol.v
always @(posedge i_clk) begin
    if(i_rst) begin
        rdState <= IDLE;
        rd_line_buffer <= 1'b0;
        o_intr <= 1'b0;
    end
    else begin
        case(rdState)
            IDLE:begin
                o_intr <= 1'b0;
                // Wait until 3 full lines (512 * 3 = 1536) are buffered
                if(totalPixelCounter >= 1536) begin
                    rd_line_buffer <= 1'b1;
                    rdState <= RD_BUFFER;
                end
            end
            RD_BUFFER:begin
                // Read out an entire row of neighborhoods
                if(rdCounter == 511) begin
                    rdState <= IDLE;
                    rd_line_buffer <= 1'b0;
                    o_intr <= 1'b1; // Trigger line complete interrupt
                end
            end
        endcase
    end
end
```

The Finite State Machine (FSM) remains in the IDLE state until the totalPixelCounter indicates that at least 1,536 pixels (three complete rows) have been successfully written to the line buffers. At this exact clock cycle, the FSM transitions to the RD_BUFFER state and asserts the rd_line_buffer signal. This signal acts as the read enable for the line buffers, causing them to simultaneously output a sliding 3x3 pixel window to the convolution unit.

4.3.3 Convolution Module (conv.v)

The Convolution Module contains the purely combinational arithmetic logic responsible for transforming the pixel data. It is designed as a multi-mode processor, capable of executing various cinematic and aesthetic filters based on parameterized selection. The module accepts a 24-bit RGB vector and outputs a modified 24-bit RGB vector.

A critical aspect of hardware arithmetic is dealing with overflow and underflow. When adding brightness or multiplying color vectors, an 8-bit register (maximum value 255) can easily overflow back to 0, causing bright white pixels to suddenly appear absolute black. To prevent this, the module utilizes strict clamping functions:

```
function [7:0] clamp_signed;
    input signed [10:0] v;
    begin
        if (v < 0)
            clamp_signed = 8'd0;
        else if (v > 255)
            clamp_signed = 8'd255;
        else
            clamp_signed = v[7:0];
        end
    endfunction
```

The module implements several filters via a case statement. For instance, the **Color Boost** mode (Mode 1) first calculates the grayscale luminance of the pixel. Because floating-point math is avoided, it uses integer multiplication and division:

$$wire [7:0] gray_cb = ((R * 30) + (G * 59) + (B * 11)) / 100$$

It then mathematically pushes the original RGB values further away from this grayscale baseline, increasing the saturation.

The **Edge Sketch** mode (Mode 9) implements an optimized 1D edge detection. Instead of a full 3x3 Sobel matrix, it stores the previous pixel's grayscale value in a register (`gray_prev`). It then calculates the absolute difference:

$$\text{wire [7:0] diff} = (\text{gray} > \text{gray_prev}) ? (\text{gray} - \text{gray_prev}) : (\text{gray_prev} - \text{gray})$$

If this difference exceeds a threshold, the output is driven to pure black; otherwise, it is driven to pure white. This creates a high-speed outline sketch of the input image.

```
// Snippet of the Convolution Core Logic
always @(posedge clk) begin
    if (rst) begin
        pixel_out_valid <= 1'b0;
        pixel_out      <= 24'd0;
    end else begin
        pixel_out_valid <= pixel_valid;
        if (pixel_valid) begin
            case (9) // Hardcoded for Edge Sketch in this simulation instance
                MODE_DUOTONE: begin
                    pixel_out <= (gray > DUOTONE_TH) ? COLOR_LIGHT :
COLOR_DARK;
                end
                MODE_EDGE_SKETCH: begin
                    pixel_out <= (diff > EDGE_TH) ? 24'h000000 : 24'hFFFFFF;
                end
                // Other modes omitted for brevity
            endcase
            prev_pixel <= pixel_in; // Store current pixel for the next clock cycle's
edge detection
        end
    end
end
end
```

4.4 Streaming Data Processing Pipeline

The integration of these modules creates a deeply pipelined streaming architecture. The lifecycle of a single pixel through this hardware pipeline is strictly deterministic and governed by the global `axi_clk`.

1. **Clock Cycle 1-1536:** The testbench asserts `pixel_valid` and begins pushing the 24-bit RGB values from the text file into the `imageProcessTop` module. The `imagecontrol` module writes these pixels sequentially into Line Buffer 0, Line Buffer 1, and Line Buffer 2. No processing occurs during this phase; the system is filling the pipeline.
2. **Clock Cycle 1537:** Three full lines are now buffered. The FSM asserts `rd_line_buffer`. Line Buffers 0, 1, and 2 simultaneously output their respective pixels.
3. **Clock Cycle 1538:** The 72-bit concatenated spatial neighborhood enters the `conv` module. The combinational logic (e.g., grayscale conversion, clamping, and edge subtraction) executes instantly within this cycle. The resulting enhanced 24-bit pixel is latched into the `pixel_out` register, and `pixel_out_valid` is driven high.
4. **Clock Cycle 1539+:** The testbench detects that `pixel_out_valid` is high and uses `$fwrite` to append the enhanced RGB vector to the `image_rgb_out.txt` file. This process continues continuously, pipelining a new pixel calculation on every subsequent clock edge until the entire 512x512 matrix is processed.

This pipelined approach ensures that the overall latency of the system is just a few clock cycles plus the initial three-line buffering time, making it exceptionally well-suited for real-time video applications.

4.5 Output Image Reconstruction

Following the completion of the Vivado simulation, the output directory contains the `image_rgb_out.txt` file. This file contains 262,144 lines of processed RGB integer values, but it lacks the necessary file headers to be recognized as a standard image by a computer's operating system.

A second Python script is employed to perform the inverse operation of the preparation phase. The script reads the output text file line by line. It initializes a new, blank PIL Image object with dimensions of 512x512 pixels. It then iterates through

the spatial coordinates (y from 0 to 511, x from 0 to 511) and populates the image array with the corresponding (R, G, B) tuple read from the text file. Finally, the `img.save()` function is called to encode the raw array into a compressed standard format (such as `output_processed.jpg` or `.png`). This reconstructed image allows for the direct visual verification and qualitative analysis of the Verilog hardware filter's performance.

```
# Python Pseudo-code for Output Image Reconstruction
from PIL import Image
def text_to_image(input_path, output_path, width=512, height=512):
    img = Image.new('RGB', (width, height))
    pixels = img.load()
    with open(input_path, 'r') as f:
        for y in range(height):
            for x in range(width):
                line = f.readline().strip()
                if line:
                    # Parse the R, G, B string into integers
                    r, g, b = map(int, line.split())
                    pixels[x, y] = (r, g, b)
    img.save(output_path)
text_to_image("image_rgb_out.txt", "output_filtered.jpg")
```

By completing this reconstruction, the experimental setup is finalized, allowing for the direct comparison of the input and output images, which forms the basis for the results and discussion presented in the subsequent chapter.

testbench executes the file read command on every rising edge of the clock, parsing the textual RGB integers, concatenating them into a 24-bit bus, and asserting the valid signal. This mechanism strictly enforces the AXI4-Stream handshaking protocol.

5.2 Timing and Control Signal Waveform Analysis

The dynamic execution of the hardware pipeline is rigorously analyzed through the XSim timing waveforms. The waveform sequence initiates with a 50-nanosecond reset assertion, during which all internal registers, read/write pointers, and FSM states are driven to zero. Upon de-assertion, the testbench begins its sequential file read operations. The valid signal transitions to high, functioning as the primary pipeline enable. As long as this signal remains high, a new 24-bit vector is ingested into the line buffers on every positive clock edge.

A critical nuance in analyzing digital waveforms at the RTL level is distinguishing between simulation events and physical output transitions. As observed in the specific waveform timing diagram, the visual changes represented on the data buses denote discrete simulation events triggered by the testbench stimulus (the reading of a new line from the text file). These stepped visual changes in the waveform viewer represent the progression of the pipeline state machine and the latching of data into intermediate registers, rather than instantaneous physical value transitions on a final output pin. The combinational logic of the spatial filters resolves instantly within the simulation delta-cycle, but the synchronous nature of the pipeline dictates that the valid output is only formally transitioned and registered on the subsequent clock edge.

The waveform provides definitive proof of the architecture's deterministic latency. For point operations, the output valid signal tracks the input valid signal with exactly one clock cycle of latency. When a hexadecimal input is presented, the corresponding mathematically transformed output appears on the output bus on the next rising edge. This confirms that the datapath operates with zero frame buffering delay, establishing a continuous throughput of one pixel per clock cycle.



Figure 2 : High-Resolution Vivado Timing Waveform highlighting pixel_valid and pixel_out_sync

5.3 Visual Results and Filter Performance Comparison

While waveform analysis validates the timing and logical integrity of the Verilog code, visual reconstruction is required to assess the qualitative efficacy of the implemented spatial domain filters. The Python post-processing script parsed the output text file to reconstruct the outputs. The baseline test subject is a 512x512 RGB image of a lion, chosen for its rich texture (fur), sharp focal planes, and complex lighting gradients.



[Figure 3 : Original 512x512 Lion Input Image]

5.3.1 Color Boost Implementation and Analysis

The Color Boost filter demonstrates the hardware's capability to perform complex, multi-channel algebraic operations concurrently. To enhance saturation, the logic must determine how far a specific color channel deviates from a pure gray tone. The hardware calculates the integer-based luminance approximation. Subsequently, the deviation for each channel is isolated, multiplied by two using an arithmetic left shift, and added back to the grayscale baseline. The visual result exhibits profoundly deepened saturation in the golden and amber

hues of the lion's mane. The critical success of this hardware implementation lies in the clamping function. By preventing integer overflow (which would cause heavily saturated colors to wrap around to black), the filter maintains structural integrity even in the brightest regions of the image.



[Figure 4 : Processed Color Boost Output Image]

5.3.2 Duotone Filter Results

The Duotone filter executes severe non-linear image segmentation. The architecture calculates an unweighted grayscale average and feeds it into a high-speed digital comparator. If the value exceeds the hardcoded threshold (128), the output multiplexer selects the light constant; otherwise, it selects the dark constant.

The output is a stark, high-contrast, two-tone representation. The filter successfully isolates the illuminated regions of the lion's face and mane from the shadowed background. This specific hardware filtering technique is highly advantageous for pre-processing in machine vision applications, where stripping out texture gradients helps downstream object-recognition algorithms identify silhouettes and bounding boxes with minimal computational effort.



[Figure 5 : Processed Duotone Output Image]

5.3.3 Edge Sketch Filter Results

Edge detection is conventionally achieved through computationally expensive 2D spatial convolution masks, which require substantial line buffering and simultaneous multiplications. To optimize hardware resources, the architecture implements a rapid 1-Dimensional edge derivative. The design utilizes a D-flip-flop register to store the exact RGB state of the previous clock cycle.

The combinational logic continuously calculates the absolute difference in luminance between the current pixel and previous pixel. If this difference is greater than the defined threshold, a sharp structural transition is identified, and the pixel is forced to pure black. The visual outcome is striking. The algorithm successfully outlines the fine hairs of the mane and the facial features while ignoring the smooth, out-of-focus background. This demonstrates that high-quality aesthetic filters can be achieved with ultra-low latency logic without relying on massive multiplier instantiation.



Figure 6 : Processed Edge Sketch Output Image

5.3.4 Gamma Correction Results

True mathematical gamma correction relies on power-law transformations, which require floating-point arithmetic and exponentiation that consume massive amounts of FPGA logic blocks. The architecture implemented herein circumvents this by utilizing a highly optimized linear approximation. The hardware shifts the input value right (effectively dividing it) and adds this fraction back to the original value.

The resulting visual reconstruction confirms the validity of this approximation. The filter successfully acts as a dynamic brightness enhancer that disproportionately benefits the shadows. While a standard linear brightness addition would wash out the bright sky background, this pseudo-gamma function smoothly lifts the dark, underexposed areas around the lion's lower jaw and chest, revealing previously obscured structural details while keeping the highlights largely intact.



[Figure 7: Processed Gamma Correction Output Image]

5.4 Quantitative Performance Analysis

Because the datapath is completely pipelined and free of iterative loops, the system is capable of accepting and processing one pixel per clock cycle continuously. Operating at a simulated frequency of 100 MHz, the theoretical throughput is 100 million pixels per second. For context, a standard 1080p high-definition video frame contains 2,073,600 pixels. At 60 frames per second, the required processing bandwidth is approximately 124 million pixels per second. By simply scaling the synthesis clock constraint to 125 MHz, this Verilog architecture can natively process uncompressed 1080p60 video in real-time, a feat that would throttle a standard CPU.

5.5 Advantages of the Proposed Architecture

The transition from software-based processing to specialized hardware logic provides several immutable advantages. By utilizing continuous line buffering rather than full-frame memory writing, the delay from the sensor input to the display output is reduced from tens of milliseconds to a mere handful of nanoseconds, which is critical for embedded vision and autonomous robotics. As proven by the XSim timing analysis, the valid and ready handshake protocol ensures that data is never dropped or overwritten. The modular separation of the memory-handling state machine from the arithmetic convolution core ensures that the system is scalable. Finally, the implementation of signed and unsigned hardware clamping functions successfully prevents the catastrophic visual artifacts associated with binary overflow and underflow, proving that the system is resilient to extreme spatial data variances.

CHAPTER 6

Conclusion and Future Scope

6.1 Conclusion

The primary objective of this project was to address the inherent computational bottlenecks associated with sequential software-based digital image processing by transitioning complex spatial filtering algorithms into dedicated digital hardware. Through the successful design, simulation, and verification of a comprehensive Verilog Hardware Description Language (HDL) architecture, this research has conclusively demonstrated the profound advantages of Field Programmable Gate Arrays (FPGAs) in high-throughput visual computing domains.

A critical achievement of this project was the development of a deeply pipelined, zero-frame-latency streaming architecture. Unlike traditional hardware implementations that rely heavily on capturing and storing entire multi-megabyte image frames into external Synchronous Dynamic Random-Access Memory (SDRAM) prior to processing, the proposed system operates strictly "on-the-fly." By ingeniously utilizing internal memory structures to create a robust Line Buffering mechanism, combined with a precise AXI-style valid/ready handshaking protocol, the architecture successfully processes continuous 24-bit RGB pixel streams. This eliminates memory access delays, reducing the end-to-end processing latency to a mere handful of clock cycles.

Furthermore, the project successfully mapped multiple complex, non-linear cinematic and aesthetic filters to the Register-Transfer Level (RTL) without relying on heavily resource-intensive floating-point arithmetic or massive multiplier instantiation. The modular conv.v core proved highly effective in executing concurrent multi-channel transformations. Specifically:

- **The Color Boost filter** successfully enhanced image vibrance by computing an integer-based grayscale luminance baseline and dynamically shifting the individual color vectors outward, all while utilizing strict hardware clamping functions to prevent binary overflow.
- **The Duotone filter** proved the system's capability for rapid, high-contrast image segmentation, providing a binary structural output highly suitable for downstream machine vision pre-processing.

- **The Edge Sketch filter** demonstrated a highly optimized approach to spatial boundary detection. By replacing the traditional computationally expensive 3x3 Sobel matrix with an ultra-low-latency 1-Dimensional absolute difference calculation against a continuously shifting `prev_pixel` register, the hardware achieved an accurate, pencil-like structural outline while drastically minimizing logic gate utilization.
- **The Gamma Correction logic** proved that power-law transformations could be effectively approximated via highly efficient bit-shifting and accumulation arithmetic, successfully recovering detail in dark, underexposed shadows without blowing out background highlights.

Quantitatively, the timing waveform analysis conducted in the Xilinx Vivado XSim environment validated the system's deterministic behavior. Operating at a simulated frequency of 100 MHz, the pipelined datapath achieved a continuous throughput of exactly one pixel per clock cycle (1 P/cc) after the initial three-row pipeline buffering delay. This translates to a processing bandwidth of 100 million pixels per second, vastly exceeding the requirements for real-time standard definition processing and establishing a robust foundation for natively processing 1080p high-definition video at 60 frames per second.

Ultimately, this project highlights the immense capability of Verilog HDL in constructing highly parallel, scalable, and power-efficient embedded vision architectures. By strictly separating the memory synchronization logic from the arithmetic datapaths, the system serves as a scalable, "future-proof" blueprint for industrial inspection systems, medical imaging accelerators, and autonomous vehicle sensor processing.

6.2 Future Scope

While the current architecture successfully achieves all primary objectives regarding spatial domain image enhancement, the inherently modular nature of the Verilog implementation provides a vast landscape for future expansion and optimization. The following avenues represent the most promising directions for scaling this technology into advanced commercial and industrial applications.

6.2.1 Integration of Real-Time Video Streaming Interfaces

The current experimental setup relies on a Python-based software-hardware co-simulation environment, utilizing `fscanf` and `fwrite` I/O tasks to emulate a camera sensor. The immediate future scope involves migrating the verified IP core from the Vivado simulator onto physical silicon (such as a Xilinx Zynq-7000 development board) to process live video data.

This transition requires the integration of high-speed hardware interfaces. The architecture must be appended with HDMI (High-Definition Multimedia Interface) or MIPI CSI-2 (Camera Serial Interface) receiver IPs. Processing live video introduces complex timing constraints, specifically the handling of Horizontal Synchronization (HSYNC), Vertical Synchronization (VSYNC), and Data Enable (DE) signals, alongside the blanking intervals inherent to video timing standards. To support this, the `imagecontrol.v` module can be upgraded to fully comply with the AXI4-Stream Video protocol, allowing it to seamlessly interface with standard Xilinx Video IP blocks (such as the Video In to AXI4-Stream IP and the AXI4-Stream to Video Out IP). By deploying the architecture on physical silicon, the system can achieve true, real-time 1080p or 4K video filtering with near-zero latency, directly outputting the enhanced stream to an external monitor.

6.2.2 Machine Learning and Adaptive Filter Integration

Currently, the filter parameters (such as the `DUOTONE_TH` edge thresholds and `BRIGHT_VAL` constants) are statically hardcoded via Verilog parameters or managed through simple multiplexer selections. The most significant leap in future capability lies in integrating Machine Learning (ML) directly onto the FPGA fabric to enable dynamically adaptive filtering.

Rather than relying on human operators to manually select a filter or adjust brightness thresholds based on changing lighting conditions, a lightweight Convolutional Neural Network (CNN) or a heuristic algorithm could be deployed in parallel to analyze the incoming video stream's histogram in real-time. For example, in an autonomous driving scenario, if the vehicle enters a tunnel, the hardware could detect the sudden drop in overall luminance and autonomously shift the `GAMMA_LEVEL` and `MODE_BRIGHTNESS` parameters on-the-fly to clarify the darkened image.

Furthermore, FPGAs are highly suited for Edge AI deployments. Future iterations of this project could expand the `conv.v` module to include hardware-accelerated Binarized Neural Networks (BNNs) or Quantized Neural Networks (QNNs). By utilizing the FPGA's dedicated Digital Signal Processing (DSP) slices, the system

could perform real-time object detection (e.g., drawing bounding boxes around pedestrians) simultaneously with the spatial image enhancement, entirely bypassing the need for a host CPU or GPU.

6.2.3 Hardware/Software Co-Design via SoC Architectures

While pure RTL design offers maximum performance, it lacks the flexibility of software for high-level decision-making and user interfacing. A major future enhancement involves deploying the Verilog image processing pipeline within a System-on-Chip (SoC) environment, such as the Xilinx Zynq architecture, which tightly couples a Dual-Core ARM Cortex-A9 processor with programmable FPGA logic.

In this hybrid paradigm, the computationally intensive, pixel-by-pixel spatial filtering (the "heavy lifting") remains in the FPGA fabric as a custom AXI Peripheral IP. Meanwhile, the ARM processor runs a lightweight embedded Linux operating system. This allows the creation of a Graphical User Interface (GUI) or a web server where an end-user can adjust sliders for brightness, color boost, or edge detection thresholds. The software running on the ARM core would then write these updated parameters directly into the FPGA's configuration registers via the AXI-Lite bus, instantly altering the behavior of the hardware pipeline without requiring recompilation or a new bitstream. This Hardware/Software co-design approach offers the ultimate balance: the deterministic, ultra-low latency of hardware convolution, combined with the infinite flexibility and ease-of-use of high-level software control.

6.2.4 Expansion to Complex 2D and Frequency Domain Filtering

Finally, the existing line-buffering architecture can be geometrically expanded. The current design utilizes a highly optimized 1D approach or a minimal 3x3 window for edge detection. Future iterations could expand the `imagecontrol.v` memory manager to instantiate 5, 7, or even 11 parallel line buffers. This expansion would provide the `conv.v` module with access to massive 5x5 or 7x7 spatial neighborhoods.

With access to larger spatial matrices, the hardware could execute highly advanced filtering techniques, such as robust Gaussian blurring for severe noise reduction, unsharp masking, or complex morphological operations (erosion and dilation) critical for medical imaging analysis. If sufficient block RAM resources are available on the target FPGA platform, the system could also be expanded to compute localized Fast Fourier Transforms (FFTs), bridging the gap into frequency domain enhancement for real-time periodic noise cancellation.

REFERENCE

- [1] M. Amiri *et al.*, “FPGA-Based Soft-Core Processors for Image Processing Applications,” *Journal of Signal Processing Systems*, vol. 87, pp. 139–156, 2017.
- [2] S. Bhavyesh *et al.*, “Parallel Pixel Processing and Filtering Architectures for Image Processing in Verilog for Real-Time Applications,” in *Proc. ICCSCE*, 2025.
- [3] A. Perri *et al.*, “A High-Performance Fully Reconfigurable FPGA-Based 2D Convolution Processor,” *Microprocessors and Microsystems*, vol. 29, no. 8–9, pp. 381–391, 2005.
- [4] J. Fernández-Berni *et al.*, “FPGA-Based Architecture for the Real-Time Computation of 2-D Convolution,” *Journal of Systems Architecture*, vol. 58, no. 8, pp. 277–285, 2012.
- [5] T. Pham-Dinh *et al.*, “An FPGA-Based Solution for Convolution Operation Acceleration,” in *Proc. ICIT*, 2022.
- [6] P. H. Panchal *et al.*, “Optimizing Image Processing with Verilog on FPGA: Techniques and Performance Enhancements,” *Journal of Semiconductor Devices and Circuits*, vol. 11, no. 3, pp. 46–53, 2024.
- [7] Q. Xu *et al.*, “A Distributed Canny Edge Detector: Algorithm and FPGA Implementation,” *IEEE Transactions on Image Processing*, vol. 23, no. 7, pp. 2944–2960, 2014.
- [8] F. Siddiqui *et al.*, “FPGA-Based Processor Acceleration for Image Processing Applications,” *Journal of Imaging*, vol. 5, no. 1, pp. 1–16, 2019.
- [9] “Efficient Hardware Implementation of 2D Convolution on FPGA for Image Processing Application,” in *IEEE Conference*, 2019.
- [10] “Dynamically Reconfigurable Parallel Architecture Implementation of 2D Convolution for Image Processing over FPGA,” in *IEEE Conference*, 2015.
- [11] “FPGA-Based Image Processing: A Comparative Study of Software-Centric and Hardware-Accelerated Approaches,” in *IEEE Conference*, 2025.
- [12] S. Aydin and H. S. Bilge, “FPGA Implementation of Image Registration Using Accelerated CNN,” *Sensors*, vol. 23, no. 14, 2023.
- [13] A. Haijoub *et al.*, “Implementing Convolutional Neural Networks on FPGA: A Survey and Research,” *ITM Web of Conferences*, vol. 52, 2023.
- [14] A. M. Sandi and G. N. S., “A Low-Latency, Area-Efficient Convolution Network for FPGA Acceleration,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 3, pp. 767–775, 2024.

Appendix A: Verilog Source Codes

APPENDIX A1

Top Module (imageProcessTop.v)

```
`timescale 1ns / 1ps
module imageProcessTop(
input  axi_clk,
input  axi_reset_n,
//slave interface
input  i_data_valid,
input [7:0] i_data,
output o_data_ready,
//master interface
output o_data_valid,
output [7:0] o_data,
input  i_data_ready,
//interrupt
output o_intr
);
wire [71:0] pixel_data;
wire pixel_data_valid;
wire axis_prog_full;
wire [7:0] convolved_data;
wire convolved_data_valid;
assign o_data_ready = !axis_prog_full;
imageControl IC(
.i_clk(axi_clk),
.i_rst(!axi_reset_n),
.i_pixel_data(i_data),
.i_pixel_data_valid(i_data_valid),
.o_pixel_data(pixel_data),
.o_pixel_data_valid(pixel_data_valid),
.o_intr(o_intr)
```



```

);
conv conv(
    .i_clk(axi_clk),
    .i_pixel_data(pixel_data),
    .i_pixel_data_valid(pixel_data_valid),
    .o_convolved_data(convolved_data),
    .o_convolved_data_valid(convolved_data_valid)
);

outputBuffer OB (
    .wr_rst_busy(),      // output wire wr_rst_busy
    .rd_rst_busy(),      // output wire rd_rst_busy
    .s_aclk(axi_clk),    // input wire s_aclk
    .s_aresetn(axi_reset_n), // input wire s_aresetn
    .s_axis_tvalid(convolved_data_valid), // input wire s_axis_tvalid
    .s_axis_tready(),    // output wire s_axis_tready
    .s_axis_tdata(convolved_data), // input wire [7 : 0] s_axis_tdata
    .m_axis_tvalid(o_data_valid), // output wire m_axis_tvalid
    .m_axis_tready(i_data_ready), // input wire m_axis_tready
    .m_axis_tdata(o_data), // output wire [7 : 0] m_axis_tdata
    .axis_prog_full(axis_prog_full) // output wire axis_prog_full
);
endmodule

```

APPENDIX A2

Control Module (imagecontrol.v)

```

`timescale 1ns / 1ps
module imageControl(
    input          i_clk,
    input          i_rst,
    input [7:0]    i_pixel_data,
    input          i_pixel_data_valid,
    output reg [71:0] o_pixel_data,
    output          o_pixel_data_valid,

```

```

output reg          o_intr
);

reg [8:0] pixelCounter;
reg [1:0] currentWrLineBuffer;
reg [3:0] lineBuffDataValid;
reg [3:0] lineBuffRdData;
reg [1:0] currentRdLineBuffer;
wire [23:0] lb0data;
wire [23:0] lb1data;
wire [23:0] lb2data;
wire [23:0] lb3data;
reg [8:0] rdCounter;
reg rd_line_buffer;
reg [11:0] totalPixelCounter;
reg rdState;

localparam IDLE = 'b0,
           RD_BUFFER = 'b1;

assign o_pixel_data_valid = rd_line_buffer;

always @(posedge i_clk)
begin
    if(i_rst)
        totalPixelCounter <= 0;
    else
        begin
            if(i_pixel_data_valid & !rd_line_buffer)
                totalPixelCounter <= totalPixelCounter + 1;
            else if(!i_pixel_data_valid & rd_line_buffer)
                totalPixelCounter <= totalPixelCounter - 1;
        end
    end
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
    begin
        rdState <= IDLE;
        rd_line_buffer <= 1'b0;
        o_intr <= 1'b0;
    end
    else
    begin
        case(rdState)
            IDLE:begin
                o_intr <= 1'b0;
                if(totalPixelCounter >= 1536)
                begin
                    rd_line_buffer <= 1'b1;
                    rdState <= RD_BUFFER;
                end
            end
            RD_BUFFER:begin
                if(rdCounter == 511)
                begin
                    rdState <= IDLE;
                    rd_line_buffer <= 1'b0;
                    o_intr <= 1'b1;
                end
            end
        endcase
    end
end

```

```

always @(posedge i_clk)
begin

```

```

    if(i_rst)
        pixelCounter <= 0;
    else
        begin
            if(i_pixel_data_valid)
                pixelCounter <= pixelCounter + 1;
            end
        end
    end
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        currentWrLineBuffer <= 0;
    else
        begin
            if(pixelCounter == 511 & i_pixel_data_valid)
                currentWrLineBuffer <= currentWrLineBuffer+1;
            end
        end
    end
end

```

```

always @(*)
begin
    lineBuffDataValid = 4'h0;
    lineBuffDataValid[currentWrLineBuffer] = i_pixel_data_valid;
end

```

```

always @(posedge i_clk)
begin
    if(i_rst)
        rdCounter <= 0;
    else
        begin

```

```

        if(rd_line_buffer)
            rdCounter <= rdCounter + 1;
        end
    end

always @(posedge i_clk)
begin
    if(i_rst)
        begin
            currentRdLineBuffer <= 0;
        end
    else
        begin
            if(rdCounter == 511 & rd_line_buffer)
                currentRdLineBuffer <= currentRdLineBuffer + 1;
            end
        end
    end
end

```

```

always @(*)
begin
    case(currentRdLineBuffer)
        0:begin
            o_pixel_data = {lb2data,lb1data,lb0data};
        end
        1:begin
            o_pixel_data = {lb3data,lb2data,lb1data};
        end
        2:begin
            o_pixel_data = {lb0data,lb3data,lb2data};
        end
        3:begin
            o_pixel_data = {lb1data,lb0data,lb3data};
        end
    end
end

```

```

        endcase
    end

    always @(*)
    begin
        case(currentRdLineBuffer)
            0:begin
                lineBuffRdData[0] = rd_line_buffer;
                lineBuffRdData[1] = rd_line_buffer;
                lineBuffRdData[2] = rd_line_buffer;
                lineBuffRdData[3] = 1'b0;
            end
            1:begin
                lineBuffRdData[0] = 1'b0;
                lineBuffRdData[1] = rd_line_buffer;
                lineBuffRdData[2] = rd_line_buffer;
                lineBuffRdData[3] = rd_line_buffer;
            end
            2:begin
                lineBuffRdData[0] = rd_line_buffer;
                lineBuffRdData[1] = 1'b0;
                lineBuffRdData[2] = rd_line_buffer;
                lineBuffRdData[3] = rd_line_buffer;
            end
            3:begin
                lineBuffRdData[0] = rd_line_buffer;
                lineBuffRdData[1] = rd_line_buffer;
                lineBuffRdData[2] = 1'b0;
                lineBuffRdData[3] = rd_line_buffer;
            end
        endcase
    end

    lineBuffer lB0(

```

```

.i_clk(i_clk),
.i_rst(i_rst),
.i_data(i_pixel_data),
.i_data_valid(lineBuffDataValid[0]),
.o_data(lb0data),
.i_rd_data(lineBuffRdData[0])
);

```

```

lineBuffer lB1(
.i_clk(i_clk),
.i_rst(i_rst),
.i_data(i_pixel_data),
.i_data_valid(lineBuffDataValid[1]),
.o_data(lb1data),
.i_rd_data(lineBuffRdData[1])
);

```

```

lineBuffer lB2(
.i_clk(i_clk),
.i_rst(i_rst),
.i_data(i_pixel_data),
.i_data_valid(lineBuffDataValid[2]),
.o_data(lb2data),
.i_rd_data(lineBuffRdData[2])
);

```

```

lineBuffer lB3(
.i_clk(i_clk),
.i_rst(i_rst),
.i_data(i_pixel_data),
.i_data_valid(lineBuffDataValid[3]),
.o_data(lb3data),
.i_rd_data(lineBuffRdData[3])
);

```

```
endmodule
```

APPENDIX A3

Line Buffer Module (linebuffer.v)

```
`timescale 1ns / 1ps
module lineBuffer(
  input  i_clk,
  input  i_rst,
  input [7:0] i_data,
  input  i_data_valid,
  output [23:0] o_data,
  input i_rd_data
);

  reg [7:0] line [511:0]; //line buffer
  reg [8:0] wrPntr;
  reg [8:0] rdPntr;

  always @(posedge i_clk)
  begin
    if(i_data_valid)
      line[wrPntr] <= i_data;
  end

  always @(posedge i_clk)
  begin
    if(i_rst)
      wrPntr <= 'd0;
    else if(i_data_valid)
      wrPntr <= wrPntr + 'd1;
  end

  assign o_data = {line[rdPntr],line[rdPntr+1],line[rdPntr+2]};
```



```

always @(posedge i_clk)
begin
    if(i_rst)
        rdPntr <= 'd0;
    else if(i_rd_data)
        rdPntr <= rdPntr + 'd1;
end

```

```

endmodule

```

APPENDIX A4

Convolution Module (conv.v)

```

module conv #(
    parameter [3:0] FILTER_MODE = 4'd0, // select filter
    parameter signed [8:0] BRIGHT_VAL = -9'sd80,
    parameter [7:0] DUOTONE_TH = 8'd128,
    parameter [23:0] COLOR_DARK = 24'h1A1A1A,
    parameter [23:0] COLOR_LIGHT = 24'hE6E6E6,
    parameter [7:0] FADE_AMOUNT = 8'd70,
    parameter [7:0] WARM_SHIFT = 8'd10,
    parameter [1:0] GAMMA_LEVEL = 2'd8,
    parameter [7:0] EDGE_TH = 8'd4
)(
    input        clk,
    input        rst,
    input        pixel_valid,
    input  [23:0] pixel_in,
    output reg    pixel_out_valid,
    output reg [23:0] pixel_out
);

// FILTER MODE DEFINITIONS
localparam MODE_BRIGHT_SOFT = 4'd0;

```

```

localparam MODE_COLOR_BOOST = 4'd1;
localparam MODE_DARK_CINEMA = 4'd2;
localparam MODE_FADE_MILK   = 4'd3;
localparam MODE_HIGH_CONTRAST = 4'd4;
localparam MODE_BRIGHTNESS  = 4'd5;
localparam MODE_DUOTONE     = 4'd6;
localparam MODE_FILM_FADE   = 4'd7;
localparam MODE_GAMMA       = 4'd8;
localparam MODE_EDGE_SKETCH = 4'd9;

```

```
// INPUT CHANNEL SPLIT
```

```

wire [7:0] R = pixel_in[23:16];
wire [7:0] G = pixel_in[15:8];
wire [7:0] B = pixel_in[7:0];

```

```
// previous pixel storage for edge filter
```

```
reg [23:0] prev_pixel;
```

```

wire [7:0] Rp = prev_pixel[23:16];
wire [7:0] Gp = prev_pixel[15:8];
wire [7:0] Bp = prev_pixel[7:0];

```

```
// COMMON FUNCTIONS
```

```

function [7:0] clamp_signed;
    input signed [10:0] v;
    begin
        if (v < 0)
            clamp_signed = 8'd0;
        else if (v > 255)
            clamp_signed = 8'd255;
        else
            clamp_signed = v[7:0];
        end
    endfunction

```

```

function [7:0] clamp_unsigned;
    input [10:0] v;
    begin
        if (v > 255)
            clamp_unsigned = 8'd255;
        else
            clamp_unsigned = v[7:0];
        end
    endfunction

```

```

function [7:0] fade_fn;
    input [7:0] x;
    reg [8:0] temp;
    begin
        temp = (x >> 1) + 9'd40;
        if (temp > 9'd255)
            fade_fn = 8'd255;
        else
            fade_fn = temp[7:0];
        end
    endfunction

```

```

function [7:0] contrast_fn;
    input [7:0] x;
    reg [8:0] temp;
    begin
        if (x < 8'd128) begin
            contrast_fn = x >> 1;
        end else begin
            temp = 9'd128 + ((x - 8'd128) << 1);
            if (temp > 9'd255)
                contrast_fn = 8'd255;
            else

```

```

        contrast_fn = temp[7:0];
    end
end
endfunction

// INTERMEDIATE CALCULATIONS

// grayscale for color boost
wire [15:0] gray_cb_full = (R * 8'd30) + (G * 8'd59) + (B * 8'd11);
wire [7:0] gray_cb      = gray_cb_full / 8'd100;

// grayscale for duotone and edge
wire [9:0] gray_duo  = R + G + B;
wire [7:0] gray      = gray_duo / 3;

wire [9:0] gray_prev_sum = Rp + Gp + Bp;
wire [7:0] gray_prev     = gray_prev_sum / 3;

wire [7:0] diff = (gray > gray_prev) ? (gray - gray_prev) : (gray_prev - gray);

// gamma
wire [8:0] Rg = {1'b0, R} + ({1'b0, R} >> GAMMA_LEVEL);
wire [8:0] Gg = {1'b0, G} + ({1'b0, G} >> GAMMA_LEVEL);
wire [8:0] Bg = {1'b0, B} + ({1'b0, B} >> GAMMA_LEVEL);

// film fade
wire [15:0] R_mul_fade = (R * FADE_AMOUNT);
wire [15:0] G_mul_fade = (G * FADE_AMOUNT);
wire [15:0] B_mul_fade = (B * FADE_AMOUNT);

wire [9:0] Rf = R + FADE_AMOUNT - (R_mul_fade / 8'd255);
wire [9:0] Gf = G + FADE_AMOUNT - (G_mul_fade / 8'd255);
wire [9:0] Bf = B + FADE_AMOUNT - (B_mul_fade / 8'd255);

wire signed [10:0] Rout = {1'b0, Rf} + $signed({1'b0, WARM_SHIFT});

```

```

wire signed [10:0] Gout = {1'b0, Gf};
wire signed [10:0] Bout = $signed({1'b0, Bf}) - $signed({1'b0, WARM_SHIFT});
// MAIN SEQUENTIAL LOGIC
always @(posedge clk) begin
    if (rst) begin
        pixel_out_valid <= 1'b0;
        pixel_out      <= 24'd0;
        prev_pixel     <= 24'd0;
    end else begin
        pixel_out_valid <= pixel_valid;

        if (pixel_valid) begin
            case (9) // controlling part
                // 0. BRIGHT + SOFT
                MODE_BRIGHT_SOFT: begin
                    pixel_out <= {
                        clamp_unsigned({3'd0, R} + 11'd30),
                        clamp_unsigned({3'd0, G} + 11'd30),
                        clamp_unsigned({3'd0, B} + 11'd30)
                    };
                end
                // 1. COLOR BOOST

                MODE_COLOR_BOOST: begin
                    pixel_out <= {
                        clamp_signed($signed({1'b0, gray_cb}) + (($signed({1'b0, R}) -
$signed({1'b0, gray_cb})) <<< 1)),
                        clamp_signed($signed({1'b0, gray_cb}) + (($signed({1'b0, G}) -
$signed({1'b0, gray_cb})) <<< 1)),
                        clamp_signed($signed({1'b0, gray_cb}) + (($signed({1'b0, B}) -
$signed({1'b0, gray_cb})) <<< 1))
                    };
                end
            end
        end
    end
end

```

```

// 2. DARK CINEMATIC
MODE_DARK_CINEMA: begin
    pixel_out <= {
        R >> 1,
        G >> 1,
        B >> 1
    };
end

// 3. FADE MILK
MODE_FADE_MILK: begin
    pixel_out <= {
        fade_fn(R),
        fade_fn(G),
        fade_fn(B)
    };
end

// 4. HIGH CONTRAST
MODE_HIGH_CONTRAST: begin
    pixel_out <= {
        contrast_fn(R),
        contrast_fn(G),
        contrast_fn(B)
    };
end

// 5. BRIGHTNESS FILTER
MODE_BRIGHTNESS: begin
    pixel_out <= {
        clamp_signed($signed({1'b0, R}) + BRIGHT_VAL),
        clamp_signed($signed({1'b0, G}) + BRIGHT_VAL),
        clamp_signed($signed({1'b0, B}) + BRIGHT_VAL)
    };
end

```

```

// 6. DUOTONE FILTER
MODE_DUOTONE: begin
    pixel_out <= (gray > DUOTONE_TH) ? COLOR_LIGHT : COLOR_DARK;
end

// 7. FILM FADE FILTER

MODE_FILM_FADE: begin
    pixel_out <= {
        clamp_signed(Rout),
        clamp_signed(Gout),
        clamp_signed(Bout)
    };
end

// 8. GAMMA FILTER
MODE_GAMMA: begin
    pixel_out <= {
        clamp_unsigned({2'd0, Rg}),
        clamp_unsigned({2'd0, Gg}),
        clamp_unsigned({2'd0, Bg})
    };
end

// 9. EDGE SKETCH FILTER
MODE_EDGE_SKETCH: begin
    pixel_out <= (diff > EDGE_TH) ? 24'h000000 : 24'hFFFFFF;
end

// DEFAULT = BYPASS
default: begin
    pixel_out <= pixel_in;
end
endcase
prev_pixel <= pixel_in;
end

```

```
    end  
  end  
endmodule
```